Architecture de Données, État Dérivé et Filtrage Dynamique

Objectifs

1. Structurer le partage d'état entre composants frères via l'ancêtre commun (*Lifting State Up*).
2. Concevoir des états dérivés (*Derived State*) performants sans dupliquer les sources de vérité dans `useState`.
3. Implémenter un moteur de filtrage et de recherche multi-critères en temps réel sur des collections d'objets.
4. Maîtriser le rendu conditionnel avancé (gestion des états vides, gardes d'affichage et prévention du piège du `0` en JSX).
5. Découper une vue métier complexe en composants de présentation (*Stateless*) et conteneurs logiques (*Stateful*).

1. Partage d'État et Communication entre Composants Frères

1.1 Le Problème de l'Isolation de l'État Local

Dans le segment précédent, nous avons vu comment doter un composant de son propre état avec `useState`. Cependant, dans une application réelle comme **OmniMonitor**, plusieurs composants distincts doivent interagir avec la même donnée :

- La **barre de filtres (`FilterBar`)** doit permettre de sélectionner un statut (`all`, `online`, `offline`) ou de saisir une recherche.
- Le **bandeau de synthèse (`SummaryBanner`)** doit afficher le nombre de stations correspondant aux critères actifs.
- La **grille de supervision (`StationGrid`)** doit afficher uniquement les cartes des stations filtrées et permettre de modifier leur statut ou de les supprimer.

En React, deux composants frères ne peuvent jamais communiquer directement entre eux. Le flux de données est strictement unidirectionnel (du parent vers l'enfant).

1.2 La Solution Architecturale : Remonter l'État (*Lifting State Up*)

Lorsqu'une donnée doit être lue ou modifiée par plusieurs composants, on applique le patron d'architecture **Lifting State Up** :

1. **Centralisation :** On retire l'état local des composants enfants et on le déclare dans leur **ancêtre commun le plus proche** (ici `App.jsx` ou `Dashboard.jsx`).
2. **Descente des données (*Props Down*) :** Le parent transmet les données brutes ou filtrées aux composants enfants via les `props`.
3. **Remontée des événements (*Events Up*) :** Le parent transmet des fonctions de rappel (*callbacks*) en `props` aux enfants (`onFilterChange`, `onToggleStatus`, `onDelete`). Lorsque l'opérateur clique dans un composant enfant, celui-ci appelle la fonction reçue pour demander au parent de mettre à jour son état.

```jsx
// Composant Parent : App.jsx
import { useState } from 'react';
import FilterBar from './components/FilterBar';
import StationList from './components/StationList';

export default function App() {
  const [stations, setStations] = useState([
    { id: 1, name: 'Toulouse-Blagnac', altitude: 152, status: 'online', alerts: 0 },
    { id: 2, name: 'Pic du Midi', altitude: 2877, status: 'offline', alerts: 3 }
  ]);
  const [statusFilter, setStatusFilter] = useState('all');

  // Callback transmis à StationList -> StationCard
  function handleToggleStatus(stationId) {
    setStations(prev =>
      prev.map(st =>
        st.id === stationId
          ? { ...st, status: st.status === 'online' ? 'offline' : 'online' }
          : st
      )
    );
  }

  return (
```

```
    <div className="container">
      {/* L'enfant FilterBar reçoit la valeur actuelle ET le callback
de modification */}
      <FilterBar
        currentFilter={statusFilter}
        onFilterChange={setStatusFilter}
      />

      {/* L'enfant StationList reçoit les données ET le callback
d'action */}
      <StationList
        stations={stations}
        onToggleStatus={handleToggleStatus}
      />
    </div>
  );
}
```

2. La Règle d'Or de l'État Dérivé (*Derived State*)

2.1 L'Anti-Pattern de la Duplication d'État

Face à un besoin de filtrage ou de calcul de statistiques (nombre de stations en ligne, moyenne d'altitude, liste filtrée par nom), une erreur classique consiste à créer plusieurs variables `useState` et à tenter de les garder synchronisées manuellement :

```jsx
// ANTI-PATTERN À PROSCRIRE : Duplication de la source de vérité
const [stations, setStations] = useState(INITIAL_STATIONS);
const [filteredStations, setFilteredStations] =
useState(INITIAL_STATIONS); // Redondant !
const [onlineCount, setOnlineCount] = useState(2); // Redondant !
```

Cette approche provoque des bugs critiques de désynchronisation : si vous supprimez une station dans `stations` en oubliant de mettre à jour `filteredStations` ou `onlineCount`, l'interface affiche des données incohérentes.

2.2 Calculer à la Volée pendant le Rendu

En React, **toute donnée qui peut être déduite ou calculée à partir d'un état existant ne doit jamais être stockée dans `useState`**.

Puisque React ré-exécute intégralement la fonction du composant à chaque fois que `stations` ou `statusFilter` change, il suffit de calculer les listes filtrées et les compteurs dans de simples constantes JavaScript (`const`) directement dans le corps du composant :

```jsx
export default function App() {
  // 1. Sources uniques de vérité (State minimal)
  const [stations, setStations] = useState(INITIAL_STATIONS);
  const [statusFilter, setStatusFilter] = useState('all');
  const [searchQuery, setSearchQuery] = useState('');

  // 2. États dérivés (recalculés automatiquement à chaque rendu)
  const filteredStations = stations.filter(station => {
    const matchesStatus = statusFilter === 'all' || station.status === statusFilter;
    const matchesSearch = station.name.toLowerCase().includes(searchQuery.toLowerCase());
    return matchesStatus && matchesSearch;
  });

  const totalCount = stations.length;
  const onlineCount = stations.filter(st => st.status === 'online').length;
  const criticalCount = stations.filter(st => st.alerts > 0).length;

  return (
    // Le JSX utilise directement filteredStations, onlineCount et criticalCount
    // ...
  );
}
```

3. Rendu Conditionnel Avancé et Pièges Classiques en JSX

3.1 Le Piège du Chiffre `0` avec l'Opérateur `&&`

Lorsqu'on souhaite afficher un badge d'alerte uniquement lorsqu'une station remonte des anomalies (`station.alerts`), on utilise fréquemment l'opérateur logique court-circuit `&&`.

En JavaScript, l'expression `a && b` retourne `a` si `a` est *falsy* (`false`, `null`, `undefined`, `""`, `0`). Or, si React ignore totalement `false`, `null` et `undefined` dans le DOM, **il affiche bel et bien le nombre `0` sous forme de texte à l'écran !**

```jsx
// PIÈGE : Si station.alerts vaut 0, React affiche un "0" parasite dans la carte !
{station.alerts && <span className="badge-danger">{station.alerts} alertes</span>}

// BONNE PRATIQUE : Toujours évaluer une expression booléenne stricte (> 0)
{station.alerts > 0 && <span className="badge-danger">{station.alerts} alertes</span>}
```

3.2 Clauses de Garde (*Early Return*) et États Vides (*Empty States*)

Une interface professionnelle doit toujours prévoir le cas où une liste filtrée ne retourne aucun résultat (par exemple, l'opérateur cherche une station qui n'existe pas ou filtre sur les stations hors ligne alors que tout le parc est opérationnel).

On utilise une clause de garde (*Early Return*) dans le composant de liste pour séparer clairement le cas vide du rendu nominal :

```jsx
export default function StationList({ stations, onToggleStatus, onDelete, onResetFilters }) {
  // Clause de garde : gestion propre de l'état vide
  if (stations.length === 0) {
    return (
      <div className="empty-state card">
        <p>Aucune station météo ne correspond aux critères de filtrage actuels.</p>
        <button className="btn" onClick={onResetFilters}>
          Réinitialiser les filtres
```

```
        </button>
      </div>
    );
  }

  // Rendu nominal de la grille
  return (
    <div className="info-grid">
      {stations.map(station => (
        <StationCard
          key={station.id}
          station={station}
          onToggleStatus={onToggleStatus}
          onDelete={onDelete}
        />
      ))}
    </div>
  );
}
```

4. Styles Dynamiques et Classes Conditionnelles Métier

Pour refléter visuellement la criticité des équipements dans **OmniMonitor**, on combine les gabarits de chaînes ES6 (*Template Literals*) avec des expressions conditionnelles sur l'attribut `className` ou des objets de style en ligne (`style`) :

```jsx
export default function StationCard({ station, onToggleStatus,
onDelete }) {
  const isOnline = station.status === 'online';
  const hasAlerts = station.alerts > 0;

  return (
    <article
      className={`card ${!isOnline ? 'card-offline' : ''}`}
      style={{
        borderLeft: hasAlerts ? '4px solid #dc2626' : '4px solid
#16a34a'
      }}
    >
      <div className="card-header">
```

```
        <h3>{station.name}</h3>
        <span className={`badge ${isOnline ? 'badge-online' :
'badge-offline'}`}>
          {isOnline ? 'En ligne' : 'Hors ligne'}
        </span>
      </div>

      <p>Altitude : {station.altitude} m</p>

      {hasAlerts && (
        <p style={{ color: '#dc2626', fontWeight: 600 }}>
          ⚠️ {station.alerts} anomalie(s) capteur détectée(s)
        </p>
      )}

      <div className="card-actions">
        <button className="btn" onClick={() =>
onToggleStatus(station.id)}>
          {isOnline ? 'Mettre hors ligne' : 'Relancer'}
        </button>
        <button className="btn btn-danger" onClick={() =>
onDelete(station.id)}>
          Supprimer
        </button>
      </div>
    </article>
  );
}
```

5. Synthèse

1. **Architecture Unidirectionnelle :** L'état partagé est stocké dans l'ancêtre commun (*Lifting State Up*), les données descendent via les `props` et les actions remontent via des fonctions *callbacks*.

2. **État Dérivé (*Derived State*) :** Ne stockez jamais dans `useState` des listes filtrées ou des compteurs statistiques ; calculez-les à la volée à chaque rendu à partir de la source unique de vérité.

3. **Rendu Conditionnel Sûr :** Utilisez systématiquement des comparaisons booléennes explicites (`count > 0 && ...`) pour éviter l'affichage accidentel du chiffre `0` dans le DOM.

4. **Expérience Utilisateur :** Prévoyez toujours un affichage explicite (*Empty State*) avec une action de réinitialisation lorsque les filtres ne retournent aucun élément.